

Optimizing a Fused Mixture-of-Experts Kernel for NVIDIA Blackwell (B200) GPUs

FlashInfer AI Kernel Generation Contest @ MLSys 2026 — *fused_moe* Track

Vansh Nawander

LTRC, IIIT Hyderabad

vansh.nawander@research.iiit.ac.in

Manish Shrivastava

LTRC, IIIT Hyderabad

m.shrivastava@iiit.ac.in

<https://github.com/vanshnawander/flashinfer-mlsys>

Abstract

We describe our solution to the `fused_moe` track of the FlashInfer AI Kernel Generation Contest at MLSys 2026, targeting NVIDIA B200 (Blackwell, SM100) GPUs. The task is to implement the fastest correct kernel for a Mixture-of-Experts feed-forward layer drawn from DeepSeek-V3/R1 [1], with FP8 block-scale quantized weights and a hierarchical Top-K routing scheme. **The kernel was developed end-to-end with AI assistance** (agent-assisted code generation, profiling, and iterative optimization). Our final Triton [3] implementation achieves a peak speedup of **7.43×** (sequence length $T=1$) and passes all **19/19** benchmark workloads, with 2.3–2.4× on long-context inputs ($T>10,000$). Key contributions are: (1) lossless FP8→BF16 promotion enabling BF16 tensor cores at zero precision cost; (2) a fused GEMM1+SwiGLU Triton kernel sharing the A-tile across gate and up projections; (3) a 4-operation dispatch table replacing 96 sequential GPU kernel launches; (4) route-weight fusion into the GEMM2 store epilogue eliminating one full output re-read per expert; and (5) FP8 bulk gather reducing token-data bandwidth by 4×.

1 Problem Statement

Task. The benchmark evaluates a Python-callable `kernel(...)` implementing one forward pass of a fused MoE FFN, assessed on 19 workloads ($T \in \{1, 7, 14-16, 32, 52-62, 80, 901, 11948, 14107\}$) for correctness and wall-clock time on an NVIDIA B200.

Architecture. Hidden size $H=7168$, intermediate $I=2048$; 256 total / 32 local experts; each token selects **Top-8** experts via hierarchical routing (8 groups of 32 experts, keep top-4 groups by sum-of-top-2 score, then global top-8 from the survivors). All weights use FP8 E4M3 block-scale quantization: every 128×128 tile shares one FP32 scalar, so $W_1 \in \mathbb{R}^{32 \times 4096 \times 7168}$ has a $(32, 32, 56)$ scale tensor and $W_2 \in \mathbb{R}^{32 \times 7168 \times 2048}$ has a $(32, 56, 16)$ scale tensor. Inputs are FP8; output is BF16.

Computation. $\text{out}[t] = \sum_{k=1}^8 w_k \cdot W_{2,e_k}^\top \cdot \text{SwiGLU}(W_{1,e_k}^\top x[t])$, where $w_k = \sigma(\text{logit}_{e_k}) / \sum_j \sigma(\text{logit}_{e_j}) \times \alpha$ and $\text{SwiGLU}(a, b) = a \odot \sigma(b) \cdot b$.

Hardware. B200 (SM100, Blackwell): 148 SMs, 4.5 PFLOPS FP8, 2.25 PFLOPS BF16, 180 TFLOPS TF32, 8 TB/s HBM3e, 228 KB shared memory per SM. Per-expert weight footprint: W_1 FP8 = 28 MB, W_2 FP8 =

14 MB (≈ 42 MB/expert, ≈ 1.3 GB for all 32 locals).

Theoretical roofline. Our analysis (see `notes/reference_moe/theoretical_timing_analysis.md` in the repository) shows: *small-T* ($T \leq 16$) is launch-overhead-bound (fixed cost ≈ 0.35 – 0.45 ms from ~ 13 PyTorch routing ops + dispatch); *medium-T* ($T=32$ – 80) is GEMM-compute-bound on weight reads; *large-T* ($T \geq 900$) is memory-bandwidth-bound on the 42 MB/expert weight stream. This analysis directly motivated each optimization stage.

2 Solution Overview & AI-Assisted Development

Development methodology. The entire kernel — routing logic, Triton kernels, dispatch optimization, precision analysis, and NCU-guided tuning — was developed in an **agent-assisted workflow**. An AI agent iteratively generated candidate kernels, ran them against the FlashInfer-Bench evaluation framework [4] on Modal B200 instances, analyzed benchmark outputs and NCU profiles, identified bottlenecks, and proposed the next optimization. Human oversight focused on directing search, validating precision rules, and resolving numerical failures. The codebase, notes, ideas, and all benchmark logs are available at <https://github.com/VanshNawander/accelerated-hpc>.

Architecture. The final kernel implements four stages:

Stage	Implementation
Routing (sigmoid, topk×2)	PyTorch, vectorized
Dispatch (sort, gather)	FP8 PyTorch, 4 ops
Expert FFN (GEMM1+SwiGLU, GEMM2)	Triton (BF16/TF32 TC)
Scatter-add	<code>index_add_</code>

Precision discipline. A key invariant discovered during development:

$$\underbrace{\text{FP8}}_{\text{load}} \xrightarrow[\text{lossless}]{\text{BF16 cast}} \underbrace{\text{BF16}}_{\cdot} \xrightarrow{\text{tl.dot}} \underbrace{\text{FP32}}_{\text{raw sum}} \xrightarrow{\times S_a S_w} \underbrace{\text{FP32}}_{\text{acc}} \xrightarrow{\text{cast}} \underbrace{\text{BF16}}_{\text{out}}$$

FP8 E4M3 has only 4 significant mantissa bits, fitting losslessly into BF16 (7 bits). Consequently, *any FP32-computed value* (SwiGLU activations, accumulated GEMM outputs)

must remain FP32 until the final output cast — violating this rule caused 18/19 correctness failures in one iteration (Section 5).

Optimization		milestones.	
Version	Key change	$T=1$	$T=14k$
Baseline	Correct routing; cuBLAS; 96 launches	$4.35\times$	$1.03\times$
Fused GEMM	On-the-fly tile dequant	$4.29\times$	$1.66\times$
Dispatch	96→4 launches; GEMM1+SwiGLU fused	$6.50\times$	$1.85\times$
Route epil.	Route-wt in GEMM2 epilogue	$6.14\times$	$2.35\times$
Final	FP8→BF16 TC; FP8 bulk gather	$7.43\times$	$2.35\times$

3 Kernel Design & Optimizations

3.1 O1: Dispatch Table — 96 Kernel Launches to 4

The baseline iterated over all 32 experts with `nonzero/any/nonzero` per expert, producing 96 sequential GPU kernel launches. Each PyTorch dispatch call carries $\approx 10\text{--}30\ \mu\text{s}$ of overhead; all 96 together cost $\approx 300\text{--}500\ \mu\text{s}$ per forward pass — measured to be 60% of total runtime at $T=1$ (see `notes/changelog.md`).

Fix. Replace the expert loop with a vectorized dispatch: (1) `nonzero(valid_local)` retrieves all (token, topk-slot) pairs in one operation; (2) `argsort(expert_ids)` sorts by expert ID; (3) `unique_consecutive(...)` computes per-expert boundaries; (4) a plain Python loop slices the sorted arrays (free, no CUDA launch). This reduced dispatch from $\sim 300\ \mu\text{s}$ to $< 5\ \mu\text{s}$, yielding 10–34% speedup across all workloads (peak: $4.29\times \rightarrow 6.50\times$ at $T=1$; large-T gained 10–13%).

3.2 O2: Fused GEMM1 + SwiGLU Triton Kernel

In the baseline, GEMM1 wrote a full $[T_k, 4096]$ FP32 intermediate to HBM before a separate SwiGLU kernel read it back ($\approx 8.4\ \text{MB}$ round-trip at $T_k=256$, plus one extra kernel launch per expert).

Fix. A single Triton kernel computes both gate and up projections in each K-block by sharing the same A-tile load — halving hidden-state read bandwidth. SwiGLU is applied in-register in the epilogue (`result = gate_acc  $\odot$   $\sigma$ (up_acc)  $\cdot$  up_acc`), eliminating the HBM store entirely. Tile sizes $\text{BM}=64$, $\text{BN}=\text{BK}=128$ align with the FP8 quantization block ($q=128$), so each tile consumes exactly one scalar weight scale.

3.3 O3: Lossless FP8→BF16 Promotion and BF16 Tensor Cores

Insight. Earlier iterations dequantized FP8→FP32, then truncated to BF16 before feeding the dot product, losing scale precision. Since FP8 E4M3 has only 4 significant bits and BF16 has 7, the promotion FP8→BF16 is *lossless*.

Factored post-dot scales. Rather than pre-multiplying block scales before the dot, we factor them out:

$$\underbrace{S_a \cdot S_w}_{\text{FP32, post-dot}} \cdot \sum_k a_{\text{fp8}}[k] \cdot w_{\text{fp8}}[k] \text{ vs. } \sum_k \underbrace{(a_{\text{fp8}} \cdot S_a)_{\text{bf16}}}_{\text{lossy!}} \cdot w_{\text{fp8}}[k].$$

The inner sum uses raw FP8 values in BF16 containers on *BF16 tensor cores* (2.25 PFLOPS on B200, vs. 60 TFLOPS on FP32 CUDA cores in prior versions). Scales are loaded once per K-block (one scalar per tile) and applied in FP32 after the dot — mathematically equivalent to full FP32 but $\approx 2\times$ faster. This is inspired by DeepGEMM’s [2] two-level accumulation approach, where partial sums from tensor cores are periodically promoted to higher-precision FP32 accumulators.

In Triton:

```
a_tile = tl.load(a_ptr + ...).to(tl.bfloat16) # FP8
      -> BF16 lossless
w_gate = tl.load(w_ptr + ...).to(tl.bfloat16)
raw     = tl.dot(a_tile, tl.trans(w_gate)) #
      BF16 TC -> FP32 acc
gate_acc += raw * (a_scale[:, None] * w_scale) #
      FP32 post-scale
```

GEMM1 uses BF16 TCs (`num_stages=3`, `num_warps=4`); GEMM2 (FP32 C-tile input from SwiGLU) uses TF32 TCs (`num_stages=4`, `num_warps=4`).

3.4 O4: FP8 Bulk Gather — 4× Less Bandwidth

All token-data gathers previously operated on FP32 hidden states (4 bytes/elem), requiring dequantization before gathering. Since hidden states are stored in FP8 (1 byte/elem), gathering *before* dequantization reduces gather bandwidth by 4× ($\approx 5.4\ \text{MB}$ saved per expert at $T_k=256$; $\approx 170\ \text{MB}$ across all 32 experts at $T=14107$). The gathered FP8 buffer is passed directly to the fused GEMM1+SwiGLU kernel, which handles in-tile dequantization.

3.5 O5: Route-Weight Fused into GEMM2 Epilogue

Each expert’s output must be multiplied by the normalized routing weight w_e before scatter-add. The baseline performed this as a separate element-wise multiply, re-reading the full $[T_k, 7168]$ GEMM2 output from HBM (7.3 MB at $T_k=256$; 234 MB across 32 experts per forward pass).

Fix. Load the routing weight (one scalar per output row) and apply it in-register at GEMM2 store time:

```
rw = tl.load(route_w_ptr + offs_m).to(tl.float32)
acc = acc * rw[:, None] # zero extra HBM traffic
tl.store(o_ptr + ..., acc, mask=...)
```

This is a pure in-register FP32 multiply with negligible

overhead, saving one full output read-modify-write pass per expert. At $T=14107$ this eliminated ≈ 234 MB of HBM traffic, contributing to the 21% speedup seen at large- T between the route-epilogue version and the dispatch-only version.

3.6 Threshold-Based cuBLAS Fallback

For experts with $T_k < 32$ tokens, launching Triton kernels costs more than the compute savings (kernel launch $\approx 15 \mu\text{s}$; actual compute $\approx 3.5 \mu\text{s}$ for 28 MB weight read at 8 TB/s). These cases fall back to `torch.matmul` with lazily dequantized FP32 weights (computed at most once per call via `a_fp32_cache`). cuBLAS uses TF32 tensor cores (180 TFLOPS) and produces more numerically stable results for small batches where Triton K-loop accumulation order differs from cuBLAS tree reduction.

4 Performance Analysis

Table 1: Final kernel on B200 (19/19 workloads passed).

T	Time (ms)	Speedup	T	Time (ms)	Speedup
1	1.390	7.43×	62	7.65	1.67×
7	2.840	3.81×	80	12.08	1.23×
15	2.288	4.69×	901	12.45	1.59×
16	5.276	2.21×	11948	14.89	2.32×
32	8.703	1.48×	14107	18.54	2.35×

Small- T ($T \leq 16$): launch-overhead bound. Our theoretical roofline analysis predicts a minimum runtime of ≈ 0.35 ms for $T=1$ (routing ops alone: 13 PyTorch calls $\times$ 15–20 $\mu\text{s} \approx 200 \mu\text{s}$ fixed cost). The actual gap between theoretical and measured reflects Python interpreter overhead and CUDA driver latency. The dispatch table removed $\approx 300 \mu\text{s}$ pushing $T=1$ from 4.29 $\times$ to 6.50 $\times$; BF16 tensor cores then doubled GEMM1 FLOP rate from ~ 60 TFLOPS to ~ 120 TFLOPS (effective), raising it further to 7.43 $\times$. The remaining gap to theoretical minimum ($\approx 2\times$) requires fusing routing into a single Triton kernel to remove the $\sim 200 \mu\text{s}$ fixed routing cost.

Medium- T ($32 \leq T \leq 80$): GEMM-bound. Each expert reads ≈ 42 MB of FP8 weights. Theoretical GEMM compute for $T_k=64$ is ≈ 0.063 ms for GEMM1 and 0.031 ms for GEMM2. At 32 experts sequentially, this alone is ≈ 3 ms; adding kernel launch overhead ($3 \times 15 \mu\text{s} \times 32 = 1.4$ ms) yields ≈ 4.4 ms theoretical vs. our ≈ 8.7 –12 ms actual. The $\approx 2\times$ gap is a known consequence of non-100% SM occupancy and weight-tile loading stalls. Native FP8 tensor cores (`tcgen05.umma`, 4.5 PFLOPS) would reduce GEMM time by $\approx 2\times$.

Large- T ($T \geq 900$): bandwidth bound. The memory-bound regime benefits most from our bandwidth optimizations. FP8 bulk gather saves 4 $\times$ on token reads; route-weight epilogue saves 234 MB of output re-reads; fused GEMM avoids materializing 3.6 GB of FP32 weights (vs. baseline). Together these move the large- T speedup from 1.03 $\times$ (baseline) to 2.35 $\times$ (final).

Memory traffic budget ($T_k=256$, per expert).

Operation	Traffic	Operation	Traffic
FP8 token gather	1.8 MB	W_2 FP8 read	14 MB
W_1 FP8 read	28 MB	GEMM2 C-tiles	29 MB
FP32 output write	7.3 MB	Total	≈ 80 MB

Dominant costs are weight reads and GEMM2 C-tile stream. The route-weight epilogue eliminated the 7.3 MB output re-read; FP8 gather saved 5.4 MB vs. FP32 gather.

Tools used for profiling. FlashInfer-Bench’s `flashinfer.bench.run_ncu` API was used to collect Nsight Compute profiles. Key metrics examined: achieved occupancy (typically 25–50% for GEMM tiles), memory bandwidth utilization ($\sim 70\%$ for large- T), and SM efficiency. These guided our tile-size choices (BM=64 vs. 128 tradeoff) and pipeline-depth settings.

5 Failed Experiments & Lessons

BF16 SwiGLU intermediate (18/19 failures). We attempted to store the SwiGLU output in BF16 to halve the GEMM1 $\rightarrow$ GEMM2 transfer. SwiGLU values have full FP32 dynamic range: the gate branch can produce values up to $448 \times \text{scale}$, and the per-element BF16 truncation error of $\approx 0.13\%$ compounds over 2048 GEMM2 K-reduction steps to exceed the benchmark tolerance. **Rule derived:** BF16 is safe only for tensors that originate from FP8 (4 sig. bits $\leq$ BF16’s 7); any FP32-computed value must remain FP32 until the final cast.

FP32 A-tiles with 4 pipeline stages (SMEM overflow). With FP32 A-tiles, each stage required $32 \times 128 \times 4 = 16$ KB for A plus 16 KB for B, so 4 stages = 128 KB plus compiler SMEM for accumulators, exceeding the 228 KB B200 limit. **Fix:** FP8 A-tiles (1 byte/element) reduce per-stage SMEM 4 $\times$, safely accommodating 3–4 stages.

Transposed weight loads (2 $\times$ slowdown). Loading weights by swapping the (N, K) strides to avoid an explicit `tl.trans()` call produced strided HBM accesses with $\approx 32\times$ cache-line amplification. **Fix:** coalesced native (N, K) loads + explicit `tl.trans(w)` before the dot.

triton.autotune (no net gain). We added `@triton.autotune` with 7 tile configurations. The autotune warmup overhead (one full benchmark run per config) dominated the measured time, and the selected config was identical to our hand-tuned BM=64, BN=BK=128. We removed autotune and hardcoded the values.

CUDA C++ kernel exploration. An ATen-based CUDA kernel was developed to eliminate Python overhead. However, matching the reference’s block-scale FP8 dequant exactly proved error-prone (ATen’s `.to(kFloat)` for FP8 applies a different scale layout), resulting in numerical failures on larger workloads. The Triton path achieved better accuracy, a faster iteration cycle, and ultimately competitive performance.

Future directions. Three high-impact optimizations remain unexplored within the contest timeframe: (1) *Native FP8 tensor cores* (`tcgen05.umma`, SM100): direct FP8 MMA with block scales from Tensor Memory at 4.5 PFLOPS — $2\times$ the current BF16 rate — estimated to yield $2.5\text{--}3\times$ on medium-T workloads. (2) *Persistent grouped GEMM*: a single GPU kernel processes all experts, eliminating the Python loop and ≈ 32 per-expert kernel launches (saves ≈ 1.4 ms fixed cost). (3) *Fused Triton routing kernel*: replacing ≈ 13 PyTorch routing ops with a single Triton kernel removes the $\approx 200\text{ }\mu\text{s}$ fixed overhead dominating small-T workloads. Together, these three are estimated to improve medium/large-T speedup by $3\text{--}5\times$ beyond the current design.

References

- [1] DeepSeek-AI. DeepSeek-V3 technical report, 2024. <https://arxiv.org/abs/2412.19437>
- [2] DeepSeek-AI. DeepGEMM: Clean and efficient FP8 GEMM kernels with fine-grained scaling, 2025. <https://github.com/deepseek-ai/DeepGEMM>
- [3] P. Tillet, H. T. Kung, and D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *MAPL*, 2019.
- [4] FlashInfer-AI. FlashInfer-Bench: Evaluation framework for GPU kernels. <https://bench.flashinfer.ai>, 2024.
- [5] NVIDIA. NVIDIA Blackwell architecture technical brief, 2024. <https://www.nvidia.com/en-us/data-center/technologies/blackwell-architecture/>
- [6] J. Shah et al. FlashAttention-4: Triton on Blackwell, 2025. <https://github.com/Dao-AILab/flash-attention>